

Data Analysis with Python 2024–2025

Parte 2: NumPy (Parte 1)

Conteúdo Teórico

Tradução e Adaptação: University of Helsinki — MOOC.fi

Nota Importante

Este material é uma tradução e adaptação baseada no curso *Data Analysis with Python 2024-2025*, oferecido pela **Universidade de Helsinque (MOOC.fi)**.

O que você vai aprender nesta página

- Saber o que é o NumPy.
- Entender por que o NumPy é eficiente.
- Saber criar, remodelar, acessar, combinar, dividir e agregar *arrays*.

Conteúdo

1 Introdução

O NumPy é uma biblioteca Python para trabalhar com *arrays* multidimensionais. Ele contém tanto as estruturas de dados necessárias para armazenar e acessar esses arrays quanto as operações e funções para realizar cálculos usando-os. Embora os arrays sejam normalmente usados para armazenar números, outros tipos de dados também podem ser armazenados, como strings. Diferentemente das listas do Python puro, a estrutura de dados fundamental do NumPy, o *array*, precisa ter o mesmo tipo de dado em todos os seus elementos. Essa homogeneidade permite a criação de funções altamente otimizadas, que recebem e retornam arrays.

Há vários usos para arrays de alta dimensão em análise de dados. Por exemplo, eles podem ser usados para:

- armazenar matrizes, resolver sistemas de equações lineares, encontrar autovalores/auto-vetores, encontrar decomposições de matrizes e resolver outros problemas conhecidos da álgebra linear;
- armazenar dados de medições multidimensionais. Por exemplo, um elemento `a[i,j]` em um array bidimensional pode armazenar a temperatura t_{ij} medida nas coordenadas i,j de uma superfície bidimensional;
- representar imagens e vídeos:
 - uma imagem em escala de cinza pode ser representada como um array bidimensional;
 - uma imagem colorida pode ser representada como um array tridimensional, em que a terceira dimensão contém os componentes de cor vermelho, verde e azul;
 - um vídeo colorido pode ser representado como um array quadridimensional;
- uma tabela bidimensional pode armazenar uma sequência de amostras, cada uma dividida em características (*features*). Por exemplo, poderíamos medir as condições climáticas uma vez por dia, incluindo temperatura, direção e velocidade do vento, e quantidade de chuva. Teríamos, então, uma amostra por dia, e as características seriam temperatura, vento e chuva. Na representação padrão desse tipo de dado tabular, as linhas correspondem às amostras e as colunas às características. Veremos mais desse tipo de dado nos capítulos sobre Pandas e Scikit-learn.

Neste capítulo, veremos:

- criação de arrays;
- tipos e atributos de arrays;
- acesso a arrays com indexação e *slicing*;
- remodelagem (*reshaping*) de arrays;
- combinação e divisão de arrays;
- operações rápidas em arrays;
- agregações de arrays;
- regras das operações binárias entre arrays;
- operações matriciais conhecidas da álgebra linear.

Começamos importando a biblioteca NumPy, usando a abreviação padrão `np`:

```
import numpy as np
```

2 Criação de arrays

Existem várias formas de criar arrays NumPy. Uma delas é passar uma lista (possivelmente aninhada) como parâmetro ao construtor `array`:

```
np.array([1,2,3])    # array unidimensional
# array([1, 2, 3])
```

Note que omitir os colchetes na expressão acima, ou seja, chamar `np.array(1,2,3)`, resulta em erro.

Um array bidimensional pode ser criado listando as linhas do array:

```
np.array([[1,2,3], [4,5,6]])
# array([[1, 2, 3],
#        [4, 5, 6]])
```

Da mesma forma, um array tridimensional pode ser descrito como uma lista de listas de listas:

```
np.array([[[1,2], [3,4]], [[5,6], [7,8]]])
# array([[[1, 2],
#         [3, 4]],
#        [[5, 6],
#         [7, 8]]])
```

Existem também algumas funções auxiliares para criar tipos comuns de arrays:

```
np.zeros((3,4))
# array([[ 0.,  0.,  0.,  0.],
#        [ 0.,  0.,  0.,  0.],
#        [ 0.,  0.,  0.,  0.]])
```

Para especificar que os elementos são `int`, em vez de `float`, usa-se o parâmetro `dtype`:

```
np.zeros((3,4), dtype=int)
# array([[0, 0, 0, 0],
#        [0, 0, 0, 0],
#        [0, 0, 0, 0]])
```

De forma semelhante, `ones` inicializa todos os elementos com o valor um, `full` inicializa todos os elementos com um valor especificado, e `empty` deixa os elementos sem inicialização:

```
np.ones((2,3))
# array([[ 1.,  1.,  1.],
#        [ 1.,  1.,  1.]])

np.full((2,3), fill_value=7)
# array([[7, 7, 7],
#        [7, 7, 7]])

np.empty((2,4))
# valores nao inicializados (lixo de memoria)
```

A função `eye` cria a matriz identidade, ou seja, uma matriz cujos elementos na diagonal valem um, e os demais valem zero:

```
np.eye(5, dtype=int)
# array([[1, 0, 0, 0, 0],
#        [0, 1, 0, 0, 0],
#        [0, 0, 1, 0, 0],
```

```
#      [0, 0, 0, 1, 0],
#      [0, 0, 0, 0, 1]])
```

A função `arange` funciona como a função `range`, mas produz um array em vez de uma lista:

```
np.arange(0,10,2)
# array([0, 2, 4, 6, 8])
```

Para intervalos com números não inteiros, é preferível usar `linspace`:

```
np.linspace(0, np.pi, 5)    # intervalo com 5 elementos igualmente
                             espacados
# array([ 0. , 0.78539816, 1.57079633, 2.35619449, 3.14159265])
```

Com `linspace` não é preciso calcular o tamanho do passo: em vez disso, especifica-se a quantidade desejada de elementos. Por padrão, o ponto final é incluído no resultado – diferentemente do que ocorre com `arange`.

2.1 Arrays com elementos aleatórios

Para testar nossos programas, podemos usar dados reais como entrada. Porém, dados reais nem sempre estão disponíveis, e coletá-los pode levar tempo. Podemos, em vez disso, gerar números aleatórios como substituto. Eles podem ser gerados facilmente com o NumPy, a partir de diversas distribuições diferentes – mencionamos apenas algumas a seguir. Dados aleatórios podem simular dados reais melhor do que, por exemplo, intervalos regulares ou arrays de valores constantes. Às vezes também precisamos de números aleatórios para escolher um subconjunto de dados reais (amostragem). O NumPy pode produzir facilmente arrays com o formato desejado preenchidos com números aleatórios. Alguns exemplos a seguir.

```
np.random.random((3,4))
# elementos distribuidos uniformemente no intervalo semiaberto [0.0,
# 1.0)

np.random.normal(0, 1, (3,4))
# elementos com distribuicao normal, media 0 e desvio padrao 1

np.random.randint(-2, 10, (3,4))
# elementos inteiros distribuidos uniformemente no intervalo [-2, 10)
```

Às vezes é útil poder recriar exatamente os mesmos dados a cada execução do programa. Por exemplo, se há um erro no programa que só se manifesta com determinada entrada, depurar o programa exige que ele se comporte de forma determinística. Podemos gerar números aleatórios de forma determinística se sempre começarmos a partir do mesmo ponto de partida. Esse ponto de partida costuma ser um número inteiro, chamado de *seed* (semente). Exemplo de uso:

```
np.random.seed(0)
print(np.random.randint(0, 100, 10))
print(np.random.normal(0, 1, 10))
```

Se você executar essa célula várias vezes, ela sempre produzirá os mesmos números – diferentemente dos exemplos anteriores. A chamada a `np.random.seed` inicializa o gerador de números aleatórios *global*. As chamadas `np.random.random`, `np.random.normal` etc. usam todas esse gerador global. Também é possível criar novos geradores de números aleatórios, e usá-los para amostrar números de uma distribuição:

```
novo_gerador = np.random.RandomState(seed=123)    # RandomState e uma
                                                    classe;

                                                    # passamos a seed ao
                                                    construtor
```

```
novo_gerador.randint(0, 100, 10)
```

Esses recursos serão usados mais adiante no material e nos exercícios, justamente para que possamos combinar quais são os dados de entrada aleatórios. Afinal, como poderíamos concordar se um resultado está correto ou não, se não pudermos concordar sobre qual é a entrada?

3 Tipos e atributos de arrays

Um array possui vários atributos: **ndim** informa o número de dimensões, **shape** informa o tamanho em cada dimensão, **size** informa o número total de elementos, e **dtype** informa o tipo dos elementos. Vamos criar uma função auxiliar para explorar esses atributos:

```
def info(nome, a):
    print(f"{nome} tem dim {a.ndim}, shape {a.shape}, size "
          f"{a.size}, e dtype {a.dtype}:")
    print(a)

b = np.array([[1,2,3], [4,5,6]])
info("b", b)
# b tem dim 2, shape (2, 3), size 6, e dtype int64:
# [[1 2 3]
#  [4 5 6]]

c = np.array([b, b])    # cria um array tridimensional
info("c", c)
# c tem dim 3, shape (2, 2, 3), size 12, e dtype int64:
# [[[1 2 3]
#    [4 5 6]]
#  [[1 2 3]
#    [4 5 6]]]

d = np.array([[1,2,3,4]])    # um vetor-linha
info("d", d)
# d tem dim 2, shape (1, 4), size 4, e dtype int64:
# [[1 2 3 4]]
```

Observe como o Python imprimiu o array tridimensional acima. As regras gerais para imprimir um array de n dimensões como uma lista aninhada são:

- a última dimensão é impressa da esquerda para a direita;
- a penúltima dimensão é impressa de cima para baixo;
- as demais também são impressas de cima para baixo, e cada “fatia” é separada da próxima por uma linha em branco.

4 Indexação, *slicing* e remodelagem

4.1 Indexação

Um array unidimensional se comporta como uma lista do Python:

```
a = np.array([1,4,2,7,9,5])
print(a[1])    # 4
print(a[-2])   # 9
```

Para um array multidimensional, o índice é uma tupla separada por vírgulas, em vez de um único inteiro:

```
b = np.array([[1,2,3], [4,5,6]])
print(b)
print(b[1,2])      # linha de indice 1, coluna de indice 2 -> 6
print(b[0,-1])     # linha de indice 0, coluna de indice -1 -> 3

# assim como nas listas, a modificacao por indexacao e possivel
b[0,0] = 10
print(b)
# [[10  2  3]
#  [ 4  5  6]]
```

Se fornecermos apenas um índice a um array multidimensional, esse índice se refere à primeira dimensão do array, ou seja, às linhas:

```
print(b[0])        # primeira linha
print(b[1])        # segunda linha
```

4.2 Slicing

O *slicing* funciona de forma parecida com as listas, mas agora podemos ter fatias em diferentes dimensões:

```
print(a)           # [1 4 2 7 9 5]
print(a[1:3])      # [4 2]
print(a[::-1])     # inverte o array: [5 9 7 2 4 1]

print(b)
print(b[:,0])      # [10  4]
print(b[0,:])      # [10  2  3]
print(b[:,1:])     # [[2 3]
                  #  [5 6]]
```

É possível até atribuir a uma fatia:

```
b[:,1:] = 7
print(b)
# [[10  7  7]
#  [ 4  7  7]]
```

Um padrão comum é extrair linhas ou colunas de um array:

```
print(b[:,0])      # primeira coluna
print(b[1,:])      # segunda linha
```

4.3 Remodelagem (*reshaping*)

Quando um array é remodelado, o número de elementos permanece o mesmo, mas eles são reinterpretados em um novo formato. Um exemplo disso é interpretar um array unidimensional como um array bidimensional:

```
a = np.arange(9)
anovo = a.reshape(3,3)
info("anovo", anovo)
# anovo tem dim 2, shape (3, 3), size 9, e dtype int64:
# [[0 1 2]
#  [3 4 5]]
```

```
# [6 7 8]]
info("a", a)
# a tem dim 1, shape (9,), size 9, e dtype int64:
# [0 1 2 3 4 5 6 7 8]

d = np.arange(4)           # array 1d
d1 = d.reshape(1,4)        # vetor-linha
dc = d.reshape(4,1)        # vetor-coluna
info("d", d)
info("d1", d1)
info("dc", dc)
```

Uma sintaxe alternativa para criar, por exemplo, vetores-coluna ou vetores-linha é usar a palavra-chave `np.newaxis`. Às vezes essa forma é mais simples ou mais natural do que usar o método `reshape`:

```
info("d", d)
info("dlinha", d[np.newaxis, :])
info("dcoluna", d[:, np.newaxis])
```

5 Concatenação, divisão e empilhamento de arrays

Existem duas formas de combinar vários arrays em um array maior: `concatenate` e `stack`. `concatenate` recebe arrays de n dimensões e retorna um array de n dimensões, enquanto `stack` recebe arrays de n dimensões e retorna um array de $n + 1$ dimensões. Alguns exemplos:

```
a = np.arange(2)
b = np.arange(2,5)
print(f"a tem shape {a.shape}: {a}")
print(f"b tem shape {b.shape}: {b}")
np.concatenate((a,b))    # concatenando arrays 1d
# array([0, 1, 2, 3, 4])

c = np.arange(1,5).reshape(2,2)
print(f"c tem shape {c.shape}: ", c, sep="\n")
np.concatenate((c,c))    # concatenando arrays 2d
# array([[1, 2],
#        [3, 4],
#        [1, 2],
#        [3, 4]])
```

Por padrão, `concatenate` junta os arrays ao longo do eixo 0. Para juntá-los horizontalmente, adicione o parâmetro `axis=1`:

```
np.concatenate((c,c), axis=1)
# array([[1, 2, 1, 2],
#        [3, 4, 3, 4]])
```

Se quisermos concatenar arrays com dimensões diferentes – por exemplo, para adicionar uma nova coluna a um array 2d –, é preciso primeiro remodelar os arrays para que tenham o mesmo número de dimensões:

```
print("Nova linha:")
print(np.concatenate((c, a.reshape(1,2))))
print("Nova coluna:")
print(np.concatenate((c, a.reshape(2,1)), axis=1))
```

Use `stack` para criar arrays de dimensão maior a partir de arrays de dimensão menor:

```

np.stack((b,b))
# array([[2, 3, 4],
#        [2, 3, 4]])

np.stack((b,b), axis=1)
# array([[2, 2],
#        [3, 3],
#        [4, 4]])

```

A operação inversa de `concatenate` é `split`. Seu argumento especifica o número de partes iguais em que o array será dividido, ou então especifica explicitamente os pontos de corte.

```

d = np.arange(12).reshape(6,2)
print("d:")
print(d)
d1, d2 = np.split(d, 2)
print("d1:")
print(d1)
print("d2:")
print(d2)

d = np.arange(12).reshape(2,6)
print("d:")
print(d)
partes = np.split(d, (2,3,5), axis=1)
for i, p in enumerate(partes):
    print("parte %i:" % i)
    print(p)

```

6 Computação rápida com funções universais

Além de fornecer uma forma de armazenar e acessar arrays multidimensionais, o NumPy também oferece diversas rotinas para realizar cálculos sobre eles. Uma das razões da popularidade do NumPy é que esses cálculos podem ser muito eficientes – muito mais eficientes do que o que o Python normalmente consegue fazer. Os maiores gargalos de eficiência são os laços, que podem ser iterados milhões, bilhões, ou até mais vezes, e por isso devem ser o mais eficientes possível. O que torna os laços lentos em Python é o fato de ser uma linguagem dinamicamente tipada: a cada expressão, o Python precisa descobrir os tipos dos argumentos das operações. Considere o seguinte laço:

```

L = [1, 5.2, "ab"]
L2 = []
for x in L:
    L2.append(x*2)
print(L2)
# [2, 10.4, 'abab']

```

A cada iteração desse laço, o Python precisa descobrir o tipo da variável `x` – que, neste exemplo, pode ser um `int`, um `float` ou uma `string` – e, dependendo desse tipo, chamar uma função diferente para realizar a “multiplicação” por dois. O que torna o NumPy eficiente é a exigência de que cada elemento em um array seja do mesmo tipo. Essa homogeneidade permite criar *operações vetorizadas*, que não operam sobre elementos individuais, mas sobre arrays (ou subarrays) inteiros. O exemplo anterior, usando operações vetorizadas do NumPy:

```

a = np.array([2.1, 5.0, 17.2])
a2 = a*2

```

```
print(a2)
# [ 4.2 10. 34.4]
```

Como cada iteração usa operações idênticas – só os dados mudam –, isso pode ser compilado para código de máquina e executado de uma só vez, evitando assim a tipagem dinâmica do Python. O nome “operação vetorial” vem da álgebra linear, em que a soma de dois vetores $v = (v_1, v_2, \dots, v_d)$ e $w = (w_1, w_2, \dots, w_d)$ é definida elemento a elemento como $v + w = (v_1 + w_1, v_2 + w_2, \dots, v_d + w_d)$.

Além da soma, várias outras funções matemáticas estão definidas em sua forma vetorial. As operações aritméticas básicas são:

Operação	Operador
adição	+
subtração	-
negação	-
multiplicação	*
divisão	/
divisão inteira	//
exponenciação	**
resto	%

Elas podem ser combinadas em expressões mais complexas. Um exemplo:

```
b = np.array([-1, 3.2, 2.4])
print(-a**2 * b)
# [ 4.41 -80. -710.016]
```

Diversas outras funções matemáticas também estão definidas. Alguns exemplos:

```
print(np.abs(b))
print(np.cos(b))
print(np.exp(b))
print(np.log2(np.abs(b)))
```

Na nomenclatura do NumPy, essas operações vetoriais são chamadas de *ufuncs* (*universal functions*, funções universais).

7 Agregações: máximo, mínimo, soma, média, desvio padrão e mais

Agregações nos permitem condensar as informações de um array em apenas alguns poucos números.

```
np.random.seed(0)
a = np.random.randint(-100, 100, (4,5))
print(a)
print(f"Minimo: {a.min()}, maximo: {a.max()}")
print(f"Soma: {a.sum()}")
print(f"Media: {a.mean()}, desvio padrao: {a.std()}")
```

Em vez de agregar sobre o array inteiro, também podemos agregar apenas sobre determinados eixos:

```
np.random.seed(9)
b = np.random.randint(0, 10, (3,4))
print(b)
```

```
print("Somadas colunas:", b.sum(axis=0))
print("Somadas linhas:", b.sum(axis=1))
```

A tabela a seguir resume algumas das funções de agregação mais comuns, com a função equivalente do Python puro (quando existe), a função do NumPy e o método correspondente do array:

Função Python	Função NumPy	Método NumPy
sum	np.sum	a.sum
–	np.prod	a.prod
–	np.mean	a.mean
–	np.std	a.std
–	np.var	a.var
min	np.min	a.min
max	np.max	a.max
–	np.argmin	a.argmin
–	np.argmax	a.argmax
–	np.median	–
–	np.percentile	–
any	np.any	a.any
all	np.all	a.all

Vamos medir o quanto a função `sum` do Python é mais lenta do que a equivalente do NumPy ao agregar um array:

```
a = np.arange(1000)
%timeit np.sum(a)
# 2.37 us +- 33.6 ns por loop (media +- desvio padrao de 7 execucoes,
# 100000 loops cada)

%timeit sum(a)
# 66.6 us +- 792 ns por loop (media +- desvio padrao de 7 execucoes,
# 10000 loops cada)
```

A velocidade do NumPy se deve, em parte, ao fato de que seus arrays precisam ter o mesmo tipo em todos os elementos. Essa exigência permite diversas otimizações eficientes.

8 Broadcasting

Já vimos que o NumPy permite operações realizadas elemento a elemento. Mas o NumPy também permite operações binárias que não exigem que os dois arrays tenham o mesmo formato. Por exemplo, podemos somar 4 a todos os elementos de um array com a expressão a seguir:

```
np.arange(3) + np.array([4])
# array([4, 5, 6])
```

Na verdade, como um array com apenas um elemento (digamos, 4) pode ser visto como um escalar 4, o NumPy também aceita a expressão a seguir, que é equivalente à anterior:

```
np.arange(3) + 4
# array([4, 5, 6])
```

Para ter uma ideia de quais operações são permitidas – ou seja, quais formatos de dois arrays são *compatíveis* –, é útil pensar que, antes de realizar a operação binária, o NumPy tenta “esticar” os arrays para que tenham o mesmo formato. No exemplo acima, o NumPy primeiro esticou o

array `np.array([4])` (ou o escalar 4) para o array `np.array([4,4,4])`, e só então realizou a soma elemento a elemento. No NumPy, esse “esticamento” é chamado de *broadcasting*.

Os arrays de entrada podem, obviamente, ter dimensões maiores, como mostra o próximo exemplo:

```
a = np.full((3,3), 5)
b = np.arange(3)
print("a:", a, sep="\n")
print("b:", b)
print("a+b:", a+b, sep="\n")
# a:
# [[5 5 5]
#  [5 5 5]
#  [5 5 5]]
# b: [0 1 2]
# a+b:
# [[5 6 7]
#  [5 6 7]
#  [5 6 7]]
```

Nesse exemplo, o segundo argumento foi primeiro transformado (*broadcasted*) no array

```
np.array([[0, 1, 2],
          [0, 1, 2],
          [0, 1, 2]])
```

e só então a soma foi realizada. Também pode ser que ambos os arrays de entrada precisem passar por *broadcasting*, como no exemplo a seguir:

```
a = np.arange(3)
b = np.arange(3).reshape((3,1))
info("a", a)
info("b", b)
info("a+b", a+b)
# a+b tem dim 2, shape (3, 3), size 9, e dtype int64:
# [[0 1 2]
#  [1 2 3]
#  [2 3 4]]
```

Para ver a que formato os argumentos foram convertidos antes da operação binária, pode-se usar a função `np.broadcast_arrays`:

```
a_convertido, b_convertido = np.broadcast_arrays(a,b)
info("a_convertido", a_convertido)
info("b_convertido", b_convertido)
```

Assim, os dois arrays passaram por *broadcasting*, mas de formas diferentes. A seguir, apresentamos as regras que governam o *broadcasting*:

1. todos os arrays de entrada com `ndim` menor que o `ndim` do array de maior dimensão recebem 1's à esquerda de sua forma (*shape*);
2. o tamanho de cada dimensão do formato de saída é o máximo entre os tamanhos de todas as entradas naquela dimensão;
3. uma entrada pode ser usada no cálculo se seu tamanho em uma determinada dimensão for igual ao tamanho de saída naquela dimensão, ou se for exatamente 1;
4. se uma entrada tiver tamanho 1 em uma dimensão de sua forma, o primeiro (e único) valor daquela dimensão é usado em todos os cálculos ao longo dela. Em outras palavras, o

mecanismo de avanço da *ufunc* simplesmente não avança ao longo dessa dimensão (o passo, ou *stride*, é 0 nessa dimensão).

Por fim, um exemplo de situação em que os dois arrays não são compatíveis:

```
a = np.array([1,2,3])
b = np.array([4,5])
try:
    a+b    # isso nao funciona, pois viola a regra 3 acima
except ValueError as e:
    import sys
    print(e, file=sys.stderr)
# operands could not be broadcast together with shapes (3,) (2,)
```

9 Resumo (semana 2)

- Aprendemos como expressões regulares podem ser usadas para especificar conjuntos regulares de strings:
 - sabemos verificar se uma string casa com uma expressão regular;
 - sabemos extrair trechos de texto que casam com uma expressão regular;
 - sabemos substituir trechos que casam com uma expressão regular por outra string.
- Sabemos ler (e escrever) um arquivo de texto, tanto linha a linha quanto o arquivo inteiro de uma vez.
- Também sabemos especificar a codificação do arquivo. A codificação UTF-8 é muito comum e consegue representar quase todo caractere ou símbolo de qualquer língua natural.
- Os parâmetros de um programa ficam no array `sys.argv`, e podemos retornar um valor do programa com a função `sys.exit`.
- Os fluxos de arquivo `sys.stdin`, `sys.stdout` e `sys.stderr` permitem entrada e saída textual básica de e para o programa.
- Todo valor em Python é um objeto:
 - classes são tipos de dados definidos pelo usuário, que determinam como construir instâncias (objetos) desse tipo;
 - a relação entre objetos e classes é testada com `isinstance`;
 - a relação entre classes é testada com `issubclass`.
- Exceções sinalizam situações excepcionais, não necessariamente erros – sabemos como capturar e lançar exceções.
- A eficiência do NumPy se baseia no fato de que as mesmas operações podem ser realizadas sobre os elementos de forma rápida, se todos os elementos forem do mesmo tipo. São as chamadas operações vetorizadas.
- Sabemos criar, remodelar, acessar, combinar, dividir e agregar arrays.

Referências e créditos

Este material é uma tradução e adaptação, para a língua portuguesa, do conteúdo do curso *Data Analysis with Python 2024–2025*, Capítulo 2 – “NumPy (Part 1)”, originalmente disponibilizado em inglês pela plataforma MOOC.fi:

`https://courses.mooc.fi/org/uh-cs/courses/data-analysis-with-python-2024-2025/
chapter-2/numpy`